

# 구조를 그린다 — 개발 세계의 지도

중급반 1강 · 개발 지식 시리즈 ① · 강의+숙제형 · 약 160분 (휴식 10분 포함) · 강사 진행 대본 (정금구)

## 📖 사용법

한 페이지 = 한 슬라이드. ★ KEY POINT(청록) = 반드시 전달할 한 문장 · 📄 화면(파랑) = 이 슬라이드에 띄울 것 · 💬 이렇게 말하세요(검정) = 실제 멘트 · 💡 강사 팁(노랑) · ⚠️ 주의(빨강) · 📌 참고(회색) · → 넘어가며. **각 페이지 왼쪽 위 검은 번호 = 슬라이드 번호**이며 하단에도 "슬라이드 NN"을 병기했다 (러닝시트 쪽번호는 표지 때문에 슬라이드 번호+1).

3대 원칙 — ① 개념 강의: 수강생 전원 개발이 처음이다. 모든 개념을 "처음 듣는 사람"에게 설명하듯 바닥부터. 아는 척 건너뛰지 않는다. ② 라이브 해부 금지: 에디터·실제 코드를 절대 띄우지 않는다 — 보여주는 건 슬라이드의 지도뿐. ③ 한 줄 복습: 이전 프레임워크는 한 줄로 소환만 하고 재강의하지 않는다 (개념 설명은 길게, 복습은 짧게 — 이 둘을 구분할 것).

## 📚 핵심 개념 한 줄 (강사용)

개발 지도 3층 = 지형(프론트·백·DB·API) · 골목(폴더는 왜 갈라졌나) · 문패(파일들의 신분)

4겹 설명 = 화면 → 프론트엔드 · 동작 → 백엔드 · 데이터 → DB · 연결 → API (7차 레이어 4겹의 본명 · 중급반은 11차 미수강 — 반드시 7차 기준으로 소환)

핵심 메시지 = "모든 파일에는 자리가 있고, 그 자리에는 이유가 있다" · 후렴 = "이미 다 해봤습니다. 오늘은 이름을 배웁니다"

수업 전 준비 = ① 02\_설계지 인쇄(전원) ② 04\_용어사전 인쇄(전원, 시작 때 배포) ③ 03\_구조지도\_강사원본 숙지(투영 금지) ④ 채점 세션 불필요

| 구분                                                 | 시간  | 슬라이드  |
|----------------------------------------------------|-----|-------|
| Part 0 — 오프닝: 중급반 개강 · 시리즈 · 핵심 메시지                | 12분 | 1~4   |
| Part 1 — 지형: 여정 ① · 브라우저/서버 · 프론트 · 백 · DB · API ★ | 55분 | 5~12  |
| ☕ 휴식                                               | 10분 | —     |
| Part 2 — 골목 1층: 루트 · 안 만든 파일들 ★ · 이름 정리 · 문패       | 29분 | 13~16 |
| Part 3 — 골목 2층: src · 폴더가 곧 주소 · JS vs Python      | 16분 | 17~19 |
| Part 4 — 원칙 5 ★ · 채점 버튼의 여정 ★                      | 22분 | 20~21 |
| Part 5 — 숙제 브리핑 · 클로징                              | 16분 | 22~23 |
| 부록 — 전체 지도 한 장 (질의응답용 · 폴더 대신 띄우기)                 | —   | 24    |

## 표지 — "오늘은 아무것도 만들지 않습니다"

⌚ 2분 (00-02)

### ★ KEY POINT

중급반 첫 수업 — 만드는 날이 아니라 이해하는 날. 용어사전을 지금 배포한다.

### ■ 화면

표지 — "만든다"에 취소선, "이해한다". 오른쪽에 지형·골목·문패 3층.

### 💬 이렇게 말하세요

중급반 첫 수업에 오신 걸 환영합니다. 오늘은 좀 특별한 날이에요 — 아무것도 만들지 않습니다. 대신, 여러분이 지금까지 만들어온 모든 것의 속을 전부 이해합니다. 들어가기 전에 종이 한 장씩 돌립니다 — 개발 용어 사전이에요. 오늘 낯선 단어가 꽤 나올 텐데, 하나도 못 외워도 됩니다. 이걸 시험 보는 사전이 아니라 필요할 때 찾아보는 사전이에요. 오늘 수업이 끝나면 이 사전의 단어들이 '아는 단어'로 바뀌어 있을 겁니다.

### 💡 강사 팁

용어사전을 시작할 때 배포하는 게 심리적 안전장치 — "외워야 하나"라는 긴장을 첫 1분에 제거한다. 설계지(숙제)는 지금 주지 말 것 — 슬라이드 22에서 배포.

→ 넘어가며: "그런데 왜 중급반 첫 수업이 '이해'일까요 — 10차를 떠올려보세요."

## 반전 — 10차엔 감췄고, 오늘은 펼친다

⌚ 3분 (02-05)

### ★ KEY POINT

진단받던 그 도구(채점기)가 오늘의 교재다 — 겉을 관찰하던 반에서, 속을 읽는 반으로.

### ■ 화면

"10차 — 소스를 감췄다 / 오늘 — 속 지도를 펼친다" 대비 2패널.

### 💬 이렇게 말하세요

10차 기억나시죠. 제가 프로그램을 하나 드리면서 소스는 안 드렸어요. 겉만 보고 속을 추론하는 관찰력 — 그 겉로 여러분이 이 반에 오셨습니다. 그러니까 여러분은 관찰로 뽑힌 반이에요. 오늘은 정반대로 갑니다. 여러분이 매주 점수를 받던 그 채점기 있죠 — 오늘 그 속 지도를 전부 펼칩니다. 폴더 하나하나, 파일 하나하나가 왜 거기 있는지. 진단받던 도구가 오늘의 교재가 되는 겁니다.

### 💡 강사 팁

"관찰로 뽑힌 반"이라는 소속감이 중급반의 연료 — 10차 얘기는 자랑스럽게. 단, 초급반과의 비교·서열 뉘앙스는 금지 ("반은 속도"라는 10차 원칙 유지).

→ 넘어가며: "이 반이 앞으로 뭘 하는 반이나 — 계획부터 보여드릴게요."

## 개발 지식 시리즈 — 내 앱 설계 3종 세트

⌚ 5분 (05-10)

### ★ KEY POINT

12 구조도 → 14 화면도 → 16 데이터도. 세 장 모으면 「내 앱 설계 3종 세트」 — 그다음에 진짜 만든다.

### ■ 화면

3단계 스텝(12 구조를 그린다 ♦ / 14 화면을 그린다 / 16 데이터를 그린다) + 숙제 연쇄.

### 💬 이렇게 말하세요

중급반은 당분간 강의와 숙제로 갑니다. 실습 시간에 쫓기지 않고, 개발이라는 세계의 지식을 제대로, 바닥부터 넣는 거예요. 지금 잡힌 계획은 세 번입니다. 오늘은 구조 — 개발 세계의 지도와 폴더 구조. 다음 시간은 화면 — HTML, CSS, 컴포넌트. 그다음은 데이터 — DB와 테이블. 그리고 매번 숙제가 하나씩 나갑니다. 오늘은 내 앱의 구조도, 다음엔 화면도, 그다음엔 데이터도. 세 장을 모으면 여러분 손에 '내 앱 설계 3종 세트'가 남아요. 그게 완성되면 — 그때 진짜로 만들기 시작합니다. 그러니까 오늘부터 그리는 건 전부, 나중에 진짜 만들 그 앱의 설계도입니다.

### ⚠ 주의 — 프레이밍

"3회 코스"라고 말하지 말 것 — 중급반도 계속 이어지는 반이다. "지금 잡힌 계획이 3회"로만. 매 회차가 직전 숙제 리뷰로 시작된다는 것도 여기서 못 박는다 (숙제 이탈 방지 장치).

→ 넘어가며: "오늘 수업 전체를 한 문장으로 말씀드리겠습니다."

## "모든 파일에는 자리가 있고, 그 자리에는 이유가 있다"

⌚ 2분 (10~12)

### ★ KEY POINT

메시지만 박고 바로 여정으로 — 여기서 폴더 얘기를 시작하지 않는다.

### ■ 화면

중앙 인용 + 후렴 "이미 다 해봤습니다. 오늘은 이름을 배웁니다".

### 💬 이렇게 말하세요

오늘의 문장입니다 — 모든 파일에는 자리가 있고, 그 자리에는 이유가 있다. 여러분, 바이브 코딩하면서 폴더 안에 정체 모를 파일들 많이 보셨죠. package.json, node\_modules, .env... 그거 전부 이유가 있어서 거기 있는 겁니다. 그리고 한 가지 더 — 여러분은 오늘 배울 걸 사실 전부 해봤어요. 만들었고, 배포했고, 키도 숨겨봤죠. 이미 다 해봤습니다. 오늘은 이름을 배웁니다.

### ⚠ 주의

여기서 특정 파일 설명으로 들어가면 Part 2와 중복되어 시간이 무너진다. 두 문장만 박고 3초 여운 → 바로 다음 슬라이드.

→ 넘어가며: "지도의 첫 장을 펴니다 — 여러분이 매일 하는 일에서 시작해요."

## 주소창의 여정 — 엔터 후 3초

⌚ 10분 (12-22)

### ★ KEY POINT

브라우저(손님) ↔ 서버(주방), 요청 ↔ 응답 — 이 한 왕복이 인터넷의 전부다. 서버는 그림이 아니라 '설계도'를 보낸다.

### ■ 화면

4스텝: 주소 입력 → 요청 → 서버(설계도 준비) → 응답·그리기.

### 💬 이렇게 말하세요

채점기 주소를 치고 엔터를 누릅니다. (실제로 센다) 하나, 둘, 셋 — 화면이 떴어요. 그 3초 동안 무슨 일이 있었는지, 식당으로 풀어볼게요. 여러분의 크롬은 손님입니다. 주소를 친다는 건 주문서를 쓰는 거예요 — "채점기 주세요." 그 주문서가 '요청'이 되어 서버로 날아갑니다. 서버는 24시간 문 여는 주방이에요. 그런데 여기서 중요한 것 하나 — 주방이 보내주는 건 완성된 그림이 아닙니다. 화면의 '설계도'예요. 이 설계도를 받아서 실제 그림으로 바꾸는 건 누구냐 — 여러분 손안의 브라우저입니다. 그러니까 여러분이 보는 채점기 화면은, 서버가 그린 게 아니라 여러분의 폰이 그린 거예요. 이 왕복 — 요청 보내고, 응답 받고 — 이게 인터넷의 전부입니다. 검색도, 유튜브도, 채점도, 전부 이 한 왕복의 반복이에요.

### ⚠ 주의 — 용어 수위

HTTP·DNS·IP를 입에 올리지 말 것. 손님·주방·주문서·서빙·설계도로만 간다. "더 정확한 이름이 궁금하면 용어사전에 있습니다" 한 마디면 충분.

### 💡 강사 팁

"화면은 여러분의 폰이 그린다"가 이 슬라이드의 심는 포인트 — 슬라이드 8(프론트엔드=브라우저에서 실행)과 슬라이드 9(그래서 다 보인다)의 복선이다. 이 여정은 슬라이드 21과 수미상관.

→ 넘어가며: "방금 여정의 양 끝에 두 명이 있었죠 — 소개부터 제대로 하겠습니다."

## 브라우저와 서버 — 두 대의 컴퓨터

⌚ 7분 (22-29)

### ★ KEY POINT

서버는 신비한 기계가 아니라 "응답하는 일을 맡은, 안 꺼지는 컴퓨터"다. 내 노트북도 서버가 될 수 있다.

### ■ 화면

브라우저(내 손안의 컴퓨터) / 서버(어딘가의 컴퓨터) 대비 2패널.

### 💬 이렇게 말하세요

브라우저부터. 크롬, 사파리, 엣지 — 다 들어보셨죠. 이름은 다르지만 전부 같은 직업입니다. 하는 일이 딱 두 가지예요. 주소를 요청으로 바꿔 보내는 것, 그리고 받은 설계도로 화면을 그리는 것. 손님이자 화가인 거죠. 참고로 "인터넷 = 크롬"이 아닙니다 — 크롬은 인터넷을 보는 창문이고, 창문은 여러 회사가 만들어요. 이제 서버. 여러분, 서버라고 하면 뭔가 서버실에 있는 깜빡거리는 특수 장비를 상상하시죠? 아닙니다. 서버는 그냥 컴퓨터예요. 요청을 기다렸다가 응답하는 '일'을 맡았을 뿐입니다. 여러분 노트북도 서버가 될 수 있어요 — 진짜로 됩니다. 그런데 왜 안 쓰냐? 노트북을 덮는 순간 가게가 닫히거든요. 새벽 3시에 손님이 와도 응대하려면 안 꺼지는 컴퓨터가 필요합니다. 그래서 그런 컴퓨터를 빌려주는 회사가 있어요 — 우리가 9차에 쓴 Vercel, 채점기가 사는 Railway가 그겁니다.

### 💡 강사 팁

"서버 = 그냥 컴퓨터"가 오늘 첫 번째 탈신비화 포인트 — 여기서 개발 세계가 만만해진다. "여러분 노트북도 됩니다"는 힘줘서. "그럼 왜 돈 내고 빌려요?"라는 질문이 나오면 성공 — 답은 대본 안에 이미 있다(안 꺼지니까).

→ 넘어가며: "등장인물 소개 끝 — 이제 이 세계의 지도를 네 구역으로 나눕니다."

## 큰 지형 — 레이어 4겹의 본명

⌚ 5분 (29~34)

### ★ KEY POINT

화면 → 프론트엔드 · 동작 → 백엔드 · 데이터 → DB · 연결 → API. 여기서 개요만 — 다음 네 슬라이드에서 하나씩 판다.

### ■ 화면

4겹 실명 대응 카드 4장 + 후렴 콜아웃.

### 💬 이렇게 말하세요 (한 줄 복습 포함)

7차에서 레이어 4겹 그렸었죠 — 데이터, 동작, 화면, 연결. 오늘은 그 네 칸의 본명을 배웁니다. 화면은 프론트엔드 — 방금 본 손님 쪽, 출입니다. 동작은 백엔드 — 주방 안쪽. 데이터는 DB — 꺾다 커도 남는 창고. 연결은 API — 프로그램끼리 쓰는 주문 창구예요. 지금은 이름표만 붙였습니다. 이제부터 네 구역을 한 곳씩 걸어서 지나갈 거예요 — 각각이 정확히 뭔지, 왜 필요한지.

### ⚠ 주의 — 재강의 금지 + 미리 다 풀지 않기

① 4겹 자체를 다시 가르치지 말 것 — "7차의 그 네 칸" 한 줄로 소환만 (중급반은 11차 미수강 — 반드시 7차 기준으로). ② 이 슬라이드에서 네 구역 상세를 미리 다 풀지 말 것 — 다음 네 장이 비어버린다. 이름표 붙이기 5분으로 끝는다.

→ 넘어가며: "첫 번째 구역 — 여러분 눈에 보이는 전부, 프론트엔드."



## 프론트엔드 — 훔 · 사용자의 기기에서 실행되는 전부

⌚ 7분 (34-41)

### ★ KEY POINT

프론트엔드 = 브라우저 안에서 실행되는 전부. UI는 생김새, UX는 경험. 그리고 — 프론트 코드는 전부 사용자에게 보인다.

### ■ 화면

정의 / 재료 삼형제 예고 / UI vs UX — 3항목.

### 💬 이렇게 말하세요

프론트엔드는 '어디서 실행되느냐'로 정의됩니다 — 여러분의 브라우저 안에서 실행되는 모든 것. 버튼, 글씨, 색깔, 눌렀을 때의 반응까지요. 재료는 세 가지인데 — 뼈대를 만드는 HTML, 옷을 입히는 CSS, 움직이게 하는 JavaScript. 오늘은 이름만 기억하세요, 이 삼형제가 14차의 주인공입니다. 그리고 자주 듣는 말 두 개를 정리해 드릴게요 — UI와 UX. UI는 생김새입니다. 버튼이 무슨 색인지, 어디 있는지. UX는 경험이에요. 원하는 걸 몇 번 만에 하는지, 헛갈리지는 않는지. 채점기로 예를 들면 — 점수 카드가 보기 좋다, 이건 UI 칭찬이고요. 채점 끝나자마자 등록 버튼이 바로 보여서 편하다, 이건 UX 칭찬입니다. 마지막으로 오늘 제일 중요한 사실 하나 — 프론트엔드 코드는 전부 사용자에게 보입니다. 왜냐? 아까 봤죠 — 화면은 여러분의 브라우저가 그려요. 그러려면 설계를 통째로 받아야 하고요. 받았다는 건 열어볼 수 있다는 겁니다. 그래서 프론트엔드에는 비밀을 둘 수 없어요.

### 💡 강사 팁

마지막 문장("비밀을 둘 수 없다")을 던져놓고 끝내면 다음 슬라이드(백엔드가 필요한 이유)가 저절로 열린다 — 답을 다음 장으로 미루는 클리프행어.

→ 넘어가며: "비밀을 못 두면 어디에 두냐 — 주방 안쪽입니다."

## 백엔드 — 주방 안쪽 · 왜 숨기나

⌚ 7분 (41-48)

### ★ KEY POINT

백엔드 = 서버에서 실행되는 것. 숨기는 이유 셋 — 비밀·신뢰·힘. 채점기가 폰에서 채점하지 않는 이유가 셋 다다.

### ■ 화면

이유 3카드: 비밀 / 신뢰 / 힘.

### 💬 이렇게 말하세요

백엔드는 서버에서 실행되는 것들입니다. 사용자 눈에 안 보이는 곳에서 계산하고, 판단하고, 저장을 지시해요. 그런데 왜 굳이 안 보이는 곳에서 할까요? 세 가지 이유가 있습니다. 첫째, 비밀. 우리 채점기는 Claude를 부를 때 API 키라는 열쇠를 씁니다. 이게 프론트에 있으면 — 방금 배웠죠, 프론트는 전부 보입니다 — 여러분 모두가 제 열쇠를 복사해갈 수 있어요. 채점 기준도 마찬가지예요. 미리 보이면 진단이 안 되죠. 둘째, 신뢰. 점수 계산을 여러분 폰에서 하면 어떻게 될까요? 조작할 수 있습니다. 100점으로 바꿔서 제출하면 그만이에요. 그래서 심판은 주방에 있어야 합니다. 셋째, 힘. 채점은 무거운 일이에요. 그걸 좋은 컴퓨터가 대신 하고, 여러분 폰은 결과만 받으면 됩니다. 폰이 오래된 기종이어도 채점이 잘 되는 이유죠. 정리하면 — 채점기가 여러분 폰에서 채점하지 않는 이유, 이 셋 전부입니다.

### 💡 강사 팁

"100점으로 바꿔 제출하면 그만"에서 웃음이 나면 성공 — 신뢰 문제가 몸으로 이해된 것. 이유 셋은 손가락으로 하나씩 꼽으며.

→ 넘어가며: "계산은 주방이 한다 치고 — 그 결과는 어디 남을까요? 세 번째 구역, 창고."

## DB — 참고 · 왜 그냥 파일이 아닌가

⌚ 7분 (48-55)

### ★ KEY POINT

DB = 데이터를 표(테이블)로 저장하는 전문 프로그램. 파일·엑셀과 다른 세 가지 — 동시성·검색·규칙.

### ■ 화면

"엑셀에 쓰면 안 되나요?" vs "DB — 참고 전문가" 대비 + 리더보드 표 예시.

### 💬 이렇게 말하세요

DB, 데이터베이스는 데이터를 저장하는 전문 프로그램입니다. 생김새부터 볼게요 — 표예요. 엑셀 표랑 똑같이 생겼습니다. 가로 한 줄이 기록 한 건, 세로 한 칸이 항목. 리더보드라면 닉네임, 점수, 제출 시각 — 이렇게 세 칸짜리 표인 거죠. 그럼 이런 질문이 나옵니다 — "표면 그냥 엑셀에 쓰면 안 돼요?" 좋은 질문이에요. 세 가지가 다릅니다. 첫째, 동시성. 리더보드에 스무 명이 동시에 제출하는 날 엑셀 파일이면 서로 덮어쓰면서 꼬입니다. DB는 동시에 100명이 써도 안 깨져요. 둘째, 검색. 기록이 수만 건 쌓였을 때 "닉네임 A의 최고점"을 엑셀에서 찾으려면 한참이지만 DB는 즉시 찾습니다. 셋째, 규칙. "점수 칸엔 숫자만"이라고 정해두면, 누가 글자를 넣으려 할 때 창고가 막아줘요. 그리고 제일 중요한 성질 — 앱을 꺼도, 서버가 재시작돼도, 참고는 남습니다. 그게 4 겹에서 '데이터'가 따로 한 칸인 이유예요. 참고 속을 설계하는 법은 16차에서 통째로 다룹니다.

### 💡 강사 팁

"엑셀이랑 똑같이 생겼다"로 시작하는 게 요령 — 낯선 것을 아는 것에 얹는다. 16차 예고는 한 문장만.

→ 넘어가며: "마지막 구역 — 그런데 이 참고, 대체 누가 어떻게 말을 걸까요?"

## API — 주문 창구 · 프로그램이 프로그램에게 말 거는 법

⌚ 7분 (55-62)

### ★ KEY POINT

사람은 화면으로, 프로그램은 API로 대화한다. 양식이 정해져 있어 서로 모르는 프로그램끼리도 협업한다.

### ■ 화면

주문서 양식 예시 / 세상의 API / 채점기의 두 API — 3항목.

### 💬 이렇게 말하세요

마지막 구역, API입니다. 이렇게 생각해 보세요 — 여러분이 채점기와 대화할 때는 화면을 씁니다. 버튼 누르고, 글자 넣고. 그럼 프로그램이 프로그램과 대화할 때는요? 개네는 화면이 필요 없어요. 대신 정해진 양식으로 대화합니다 — 그 양식이 API예요. 주문 창구라고 생각하면 됩니다. "이 프롬프트 채점해줘"라고 정해진 양식으로 요청하면, "점수표 여기 있습니다"라고 정해진 형식으로 답이 와요. 이게 왜 대단하냐면 — 양식만 지키면 서로 전혀 모르는 프로그램끼리도 협업할 수 있거든요. 지도 앱이 길을 아는 것도, 날씨 앱이 기온을 아는 것도, 전부 남의 API를 불러 쓰는 겁니다. 우리 채점기에는 API가 두 군데 있어요. 하나는 우리가 만든 창구 — 여러분 브라우저가 채점을 주문하는 /api/score. 또 하나는 남의 창구 — 우리 서버가 이번엔 손님이 되어서, Claude API에 채점을 주문합니다. 우리 서버가 어디서 주방이고 어디서 손님인 거죠. 그게 연결이라는 지형입니다.

### 💡 강사 팁

"우리 서버가 주방이자 손님"이 이 슬라이드의 고급 포인트 — 여기가 이해되면 8차 자동화(MCP now → API later)가 나중에 쉽게 붙는다. 다만 오늘은 그 연결까지 말하지 않는다.

→ 넘어가며: "네 구역 완주 — 이제 이 지형이 채점기 실물에 어떻게 박혀 있는지 봅니다."

## 채점기 하나에 다 있다 → ☕ 휴식

⌚ 5분 (62-67) + 휴식 10분 (67-77)

### ★ KEY POINT

Next.js = 홀+주방 한 몸 · Supabase = 창고(폴더 밖!) · Railway = 무대. 여기까지가 전반전 — 휴식 후 폴더를 연다.

### ■ 화면

Next.js / Supabase / Railway 3항목 + 9차 5요소 연결.

### 💬 이렇게 말하세요 (한 줄 복습 포함)

9차에서 살아있는 서비스 5요소 배웠죠 — 몸, 기억, 무대, 창고, 맥박. 채점기도 똑같이 살아 있습니다. 몸은 Next.js로 만들어졌어요 — 홀과 주방을 한 몸에 담은 조립 키트인데, 이름 정리는 후반전에 제대로 합니다. 창고는 Supabase. 그런데 여기서 놀라운 사실 하나 — 이 창고, 우리 폴더 안에 없습니다. 다른 회사 서버에 살아요. 우리가 가진 건 창고 열쇠와 주소뿐입니다. 그래서 '연결'이 필요한 거예요. 무대는 Railway — 아까 배운 그 '안 꺼지는 컴퓨터'를 빌려주는 곳입니다. 오늘 후반전에 지도를 그릴 대상은 이 중 '몸' 폴더예요. 자 — 여기까지가 전반전입니다. 10분 쉬고, 폴더를 엽니다.

### 💡 강사 팁

휴식 전 예고("쉬고 나면 폴더를 연다")로 후반전 기대를 걸어둔다. 휴식 중 용어사전을 뒤적이는 사람이 보이면 잘 가고 있는 것. 정시 재개 — 후반전 첫 슬라이드(13)는 트리 지도라 지각자도 따라볼기 쉽다.

→ (휴식 후) "채점기 폴더를 열었습니다 — 처음 보이는 것들부터."

## 루트 — 폴더를 열면 처음 보이는 것들

🕒 7분 (77-84)

### ★ KEY POINT

루트의 절반은 내가 만들지 않은 파일 — 도구와 관례가 만든다. src만이 "내가 만든 것의 전부".

### ■ 화면

루트 트리 지도(슬라이드에 재작성된 것). 절대 실제 탐색기·에디터를 열지 않는다.

### 💬 이렇게 말하세요

채점기 폴더를 열면 이렇게 생겼습니다. 위에서부터 훑어볼게요. package.json — 재료 목록. 설정 파일 네댓 개 — 주방 세팅. .env.local — 열쇠 서랍, 우리 API 키가 여기 삽니다. .gitignore — 문지기. README와 CLAUDE.md — 문패 두 장. docs와 public — 문서와, 그대로 내보내는 것들. node\_modules — 제일 무겁고 제일 수상한 폴더. 그리고 맨 아래 src — 여기 주목하세요. 제가 만든 코드는 전부 src 안에만 있습니다. 그 위에 있는 것들은요? 절반은 도구가 만들었고, 절반은 관례가 정해준 거예요. 지금은 이름표만 붙였습니다 — 다음 장에서 이 정체불명 파일들을 하나씩 심문할 겁니다.

### ⚠ 주의 — 라이브 해부 금지

"실제로 보여주세요"가 나와도 에디터를 열지 않는다 — "코드는 다음 시즌에 열어요. 오늘 지도부터 안 그리면 그때 길을 잃습니다." 지도(슬라이드)로만.

→ 넘어가며: "자, 심문 시작 — 안 만든 파일들의 정체."

## 내가 안 만든 파일들의 정체 — 요리로 읽기

⌚ 10분 (84-94)

### ★ KEY POINT

목록(package.json) · 참고(node\_modules) · 조리(빌드) · 열쇠(.env) · 문지기(.gitignore). 퀴즈가 오늘 최고의 아하 모먼트.

### ■ 화면

요리 비유 5항목 + 퀴즈 "node\_modules를 지우면 앱은 죽을까요?"

### 💬 이렇게 말하세요 (한 줄 복습 포함)

5차에서 자동화를 요리로 배웠으니 오늘도 요리로 갑니다. package.json은 재료 목록이에요. 이 앱의 이름, 필요한 부품, 실행 명령이 적혀 있습니다. 부품이 뭐냐면 — 남이 만들어둔 코드 꾸러미예요. 달력, 차트, DB 연결 도구... 세상 개발자들이 만들어 공개해둔 걸 우리가 갖다 쓰는 거죠. 모든 걸 직접 만들면 아무것도 못 만드니까, 바퀴는 사 오고 우리는 차를 조립하는 겁니다. 그 부품 가게가 npm이고요. npm install을 치면 — 다들 쳐보셨죠? — 목록을 보고 가게에서 부품을 사다가 참고를 채웁니다. 그 참고가 node\_modules예요. 그래서 제일 무겁습니다, 수만 개 파일이 들어 있거든요. 빌드는 조리예요. 개발자가 쓰기 편한 코드와 컴퓨터가 돌리기 좋은 코드는 형태가 달라서, npm run build로 조리해서 내놓습니다. .env는 열쇠 서랍 — 9차에서 API 키 숨길 때 써봤죠. 비밀을 코드에 직접 쓰면 GitHub에 올라가는 순간 전 세계 공개입니다. 그래서 서랍에 따로 두는 거예요. 그리고 .gitignore, 문지기 — 비밀과, 다시 만들 수 있는 것들을 GitHub 문 앞에서 막습니다. 자, 그럼 퀴즈. node\_modules를 통째로 지우면 — 이 앱은 죽을까요, 살까요? 죽는다 손! (확인) 산다 손! (확인, 3초 침묵)

### ⚠ 주의 — 3초의 침묵

손 확인 후 3초 멈추고 공개한다. 답: "삽니다. 재료 목록만 있으면 npm install 한 번으로 참고가 다시 채워져요. 그래서 문지기가 참고를 GitHub에 안 올리는 겁니다 — 목록만 있으면 되니까." — 목록·참고·문지기가 한 줄로 웨이는, 오늘 최고의 순간.

→ 넘어가며: "그런데 아까부터 Next.js, React... 이름이 많죠. 정리하고 갑시다."

## 그 많던 이름들 — 러시아 인형

🕒 7분 (94-101)

### ★ KEY POINT

언어=말 · 라이브러리=내가 부르는 부품 상자 · 프레임워크=나를 끼워 넣는 조립 키트. JS→TS, React를 Next.js가 감싼다.

### ■ 화면

JS → TS → React → Next.js 4스텝 + 러시아 인형 콜아웃.

### 💬 이렇게 말하세요

"그거 다 Next.js 아니었어?" — 하실 만합니다. 정리해드릴게요. 먼저 구분 세 가지. 언어는 코드를 쓰는 말입니다. 라이브러리는 부품 상자 — 내가 필요할 때 꺼내 쓰는 거예요. 프레임워크는 조립 키트 — 뼈대와 규칙을 키트가 정하고, 나는 빈칸을 채웁니다. 구분 요령은 — 내가 부르면 라이브러리, 나를 끼워 넣으면 프레임워크. 이제 이름 넷을 세워볼게요. JavaScript — 브라우저가 알아듣는 유일한 언어, 웹의 모국어입니다. TypeScript — 그 JS에 약속을 엮은 거예요. "이 값은 숫자다"라고 미리 약속해서 실수를 실행 전에 잡습니다. 우리 파일들이 .ts, .tsx인 이유예요. React — 화면을 부품으로 조립하는 방식을 주는 라이브러리. Next.js — React를 가지고 앱 전체를 짓게 해주는 프레임워크입니다. 주소, 서버, 빌드까지 다 챙겨주고, 폴더 규칙도 애가 정해요. 그래서 정리하면 — 채점기는 Next.js로 지은 앱이고, 그 속은 React 부품이고, 그 부품은 TypeScript로 쓰였고, TS는 결국 JavaScript입니다. 러시아 인형이에요.

### 💡 감사 팁

"내가 부르면 라이브러리, 나를 부르면 프레임워크"는 두 번 반복할 가치가 있는 문장. 여기서 시간이 밀리면 안 된다 — 깊은 얘기는 14차 뒤편을 스스로 상기.

→ 넘어가며: "루트에 남은 마지막 두 파일 — 문패."



## 문패 2장 — README와 CLAUDE.md

⌚ 5분 (101-106)

### ★ KEY POINT

README = 사람용 문패, CLAUDE.md = AI용 문패. AI는 매번 처음 출근한 직원 — 문서가 곧 AI의 기억이다.

### ■ 화면

README / CLAUDE.md 대비 2패널 + "39KB, A4 30장" 콜아웃.

### 💬 이렇게 말하세요 (한 줄 복사 포함)

6차에서 우리가 직접 만들었던 두 파일입니다 — AI한테 한 장, 사람한테 한 장. README는 사람용 문패예요. 폴더만 봐서는 이게 뭔지 모르잖아요. 그래서 "이건 이런 앱이고, 이렇게 켜다"를 적어둡니다. 이름이 대문자인 것도 관례예요 — READ ME, 나부터 읽으라고 소리치는 거죠. 6개월 뒤의 나도 '처음 온 사람'이라는 걸 잊지 마세요. CLAUDE.md는 AI용 문패입니다. 왜 필요하냐 — AI는 매번 처음 출근한 직원이거든요. 어제 일을 기억 못 해요. 수칙이 없으면 매번 같은 사고를 냅니다. 그래서 규칙과 함정과 히스토리를 문서로 적어두는 거예요. 문서가 곧 AI의 기억입니다. 채점기의 CLAUDE.md가 얼마나 되냐면 — 39킬로바이트, A4로 30장. 11개 강의의 역사가 전부 적혀 있어요. 문서도 구조의 일부입니다.

### 📌 참고

"내용 보여주세요" 하면 숫자(39KB·30장)까지만 — 내용 투영은 하지 않는다. "여러분도 6차에 만든 게 있죠 — 오늘 숙제에도 문패 칸이 있습니다"로 숙제 예고를 슬쩍.

→ 넘어가며: "1층 끝 — 이제 src, 내가 만든 것의 동네로."

## src — 내가 만든 것의 다섯 골목

🕒 7분 (106-113)

### ★ KEY POINT

app(주소 있는 동네) · components(부품) · lib(동작) · types(계약) · data(콘텐츠). 경계는 흐릴 수 있다 — 지도는 단순화다.

### ■ 화면

src 다섯 골목 트리 + 4겹 태그.

### 💬 이렇게 말하세요 (한 줄 복습 포함)

src를 열면 골목이 다섯입니다. app — 주소가 있는 것들의 동네예요, 페이지들이 삽니다. components — 화면 부품입니다. 컴포넌트가 뭐냐면, 화면의 한 조각을 레고 부품처럼 만들어둔 거예요. 버튼, 카드, 목록 같은 것들. 왜 부품으로 만드냐 — 같은 버튼을 열 군데 복사하면 고칠 때 열 군데를 고쳐야 하잖아요. 부품이면 한 번만 고칩니다. 채점기엔 26개가 있어요 — 채점 카드 다섯 종, 결과 카드 다섯 종, 공용 버튼들. lib — 동작이에요. 채점 기준, 세션 처리, 계산하는 것들. types — 계약입니다. 2차 입출력 계약 기억나시죠 — 그게 코드가 되면 폴더 하나를 받아요. "점수는 이렇게 생겼다"를 약속해두는 곳. data — 강의 내용 데이터인데, 한 파일이 225킬로바이트예요. 이 애긴 원칙에서 다시 합니다. 그리고 고백 하나 — app 골목엔 화면도 살고 연결도 삽니다. 4겹이 폴더에 딱 떨어지지 않아요. 지도는 단순화입니다. 실물은 경계가 흐려요. 그래도 지도 없이 다니는 것보단 백배 낫습니다. 숙제 때 태그 두 개 붙여도 됩니다.

### 💡 강사 팁

"경계가 흐리다"를 숨기지 말 것 — 숨기면 숙제에서 막힌 학생이 자책한다. 컴포넌트 설명(왜 부품인가)은 여기서 처음 제대로 나오는 것 — 14차 예고를 겸하므로 건너뛰지 않는다.

→ 넘어가며: "그중 app 골목엔 아주 예쁜 규칙이 하나 있습니다."

## 폴더가 곧 주소다

⌚ 5분 (113-118)

### ★ KEY POINT

app 안의 폴더 경로 = URL (Next.js의 관례). [code]는 변하는 주소. api는 화면 없는 골목.

### ■ 화면

주소 ↔ 폴더 대응표 + 퀴즈 "채점기의 화면(주소)은 몇 개?"

### 💬 이렇게 말하세요

app 골목의 규칙 — 폴더가 곧 주소입니다. 이건 아까 배운 프레임워크, Next.js가 정한 관례예요. practice라는 폴더를 만들면 /practice라는 페이지가 생깁니다. play 안에 대괄호로 [code] — 이건 변하는 주소예요. 세션 코드마다 다른 방이 되는 거죠. /play/ABCD, /play/XYZW. 그리고 api 폴더 — 주소는 있는데 화면이 없습니다. 아까 배운 API, 주문 창구가 바로 여기 사는 거예요. 화면 없는 골목. 자, 퀴즈 — 여러분이 매주 쓰신 앱입니다. 채점기의 화면, 그러니까 주소가 모두 몇 개일까요? 폰으로 직접 눌러봐도 됩니다.

### 💡 강사 팁

유일하게 허용되는 '실물 접촉' — 폰으로 채점기 앱을 눌러보게 하는 것 (앱 화면 OK, 코드 NO). 답: 6개 (/ · /practice · /host · /play/[code] · /play/[code]/board · /install). install까지 찾은 사람은 관찰왕으로 칭찬.

→ 넘어가며: "그런데 파이썬 프로젝트를 열면 이 지도가 무용지물일까요?"

## 같은 원리, 다른 생김새 — JS와 Python

⌚ 4분 (118-122)

### ★ KEY POINT

이름은 달라도 자리는 같다 — 재료 목록·참고·소스. 오늘 배운 건 자리를 읽는 법이다.

### ■ 화면

JS(package.json·node\_modules·src) ↔ Python(requirements.txt·venv·main.py) 대응 2패널.

### 💬 이렇게 말하세요

언젠가 파이썬 프로젝트를 열게 될 거예요. 폴더가 낯설어 보이겠지만 — 자리를 찾으면 됩니다. 재료 목록? 파이썬에선 requirements.txt라고 부릅니다. 참고? venv예요. 소스? main.py나 src 폴더고요. 이름만 바뀌었지 자리는 똑같죠. 오늘 배운 건 Next.js의 지도가 아니라 자리를 읽는 법입니다. 어떤 언어의 프로젝트를 열어도 — 재료 목록 찾고, 참고 확인하고, 소스가 어딘지 보면 길을 안 잃습니다.

### ⚠ 주의

파이썬 상세(가상환경 만들기, pip 명령어)로 빠지지 말 것 — "자리는 같다" 한 문장이 전부다. 2분 만에 끝나도 좋다.

→ 넘어가며: "지도는 다 그렸습니다 — 이제 '좋은 지도'의 조건."

## 좋은 구조의 원칙 5

⌚ 12분 (122-134)

### ★ KEY POINT

①한 폴더=한 책임 ②화면·동작 분리 ③계약은 따로 ④데이터·코드 분리 ⑤문패 — 이 다섯이 숙제의 채점 기준이다.

### ■ 화면

원칙 5 카드 (④에 강조) + content.ts 증거 콜아웃.

### 💬 이렇게 말하세요 (한 줄 복습 포함)

첫째, 한 폴더 = 한 책임. 8차에서 단계 쪼갤 때 '하나의 책임' 얘기했죠 — 폴더도 같습니다. 이유를 한 줄로 못 쓰는 폴더는 없어야 할 폴더예요. 둘째, 화면과 동작을 섞지 않는다. 채점 로직이 버튼 파일 안에 살면 어떻게 되냐 — 로직을 고치다 화면이 깨지고, 화면을 고치다 로직이 깨집니다. 서로 인질이 되는 거예요. 그래서 채점기는 로직을 lib과 api에, 화면을 components에 갈라놔했습니다. 셋째, 주고받는 것은 계약으로 따로 — types 폴더. 약속이 없으면 점수 모양이 바뀔 때 어디가 깨질지 아무도 모릅니다. 약속이 있으면 어긋나는 순간 도구가 알려 줘요. 넷째 — 오늘 제일 힘줄 원칙입니다. 데이터와 코드를 섞지 않는다. 증거를 보여드릴게요. 이 채점기에 강의 11개 추가되는 동안, 대부분의 회차에서 고친 파일이 몇 개였을까요? (답 받기) ...하나입니다. content.ts. 강의 내용이 전부 데이터로 분리돼 있어서, 새 강의를 넣을 때 코드에 손을 안 대요. 구조를 잘 갈라두면 이런 힘이 생깁니다. 다섯째, 문패를 단다 — 사람용과 시용, 아까 그 두 장. 그리고 — 이 다섯이 오늘 숙제의 채점 기준입니다.

### 💡 강사 팁

원칙 4의 "몇 개였을까요?"는 반드시 답을 받고 공개 — 3~5개 예상이 대부분이라 "하나"의 임팩트가 산다. 시간이 밀리면 ②③을 압축하고 ④는 절대 줄이지 말 것.

→ 넘어가며: "이제 전부 한 여정에 꿰겠습니다 — 수백 번 누른 그 버튼으로."

## 채점 버튼 하나의 여정

⌚ 10분 (134-144)

### ★ KEY POINT

버튼(화면) → api(연결) → 채점 기준+Claude(동작·서버) → 결과(화면) → Supabase(데이터) → 리더보드(화면). .env는 서버에서만 — 16차 떡밥.

### ■ 화면

6스텝 여정 + 4겹 태그 + 열쇠 위치 콜아웃.

### 💬 이렇게 말하세요

아까는 주소로 들어왔죠 — 이번엔 버튼으로 들어갑니다. 여러분이 수백 번 누른 채점하기 버튼이요. 하나 — 버튼은 화면 부품입니다. components 골목에 살아요. 둘 — 누르는 순간 여러분의 프롬프트가 /api/score로 날아갑니다. 화면 없는 골목, 연결이죠. 셋 — 서버에서 채점 기준이 깨어납니다. lib에 있는 채점 프롬프트가 여러분의 답안과 함께 Claude에게 보내져요. 이때 열쇠가 필요합니다 — API 키. 그 열쇠는 어디 있다고요? .env, 서버 쪽에만요. 왜 서버에만 두는지 이제 설명할 수 있죠 — 프론트는 전부 보이니까요. 브라우저에 뒀다면 여러분 모두가 제 열쇠를 가져갈 수 있었을 겁니다. 넷 — 점수가 돌아오면 결과 카드가 그립니다. 다시 화면. 다섯 — 리더보드에 등록하면 Supabase 창고에 저장됩니다. 데이터. 여섯 — 리더보드 화면이 창고를 읽어 순위를 그려요. 보세요 — 버튼 하나에 홀도, 주방도, 창고도, 창구도 전부 들어 있습니다. 오늘 배운 지형 전부가 이 3초 안에 있는 거예요. 이 여정이 머릿속에 그려지면, 오늘 지도는 완성입니다.

### 💡 감사 팁

슬라이드 05와 수미상관임을 명시적으로 — "아까는 주소로, 이번엔 버튼으로". 손가락으로 스텝을 하나씩 짚으며 천천히. 오늘 수업의 완성 순간이므로 서두르지 않는다.

### ⚠️ 주의

.env 위치는 반드시 짚는다 — 16차(데이터·보안)의 핵심 떡밥. 단 탈취 원리까지는 들어가지 말 것.

→ 넘어가며: "지도는 완성 — 이제 여러분 차례입니다. 숙제 나갑니다."

## 숙제 — 내 앱 폴더 구조 설계

🕒 12분 (144~156)

### ★ KEY POINT

합격 기준 3개 — 최상위 3~6개 / 모든 폴더에 "왜" / 4겹 태그. 14차는 이 종이로 시작한다.

### ■ 화면

합격 기준 카드 3장 + 보너스. 이때 02\_설계지를 배포한다.

### 💬 이렇게 말하세요

설계지 나갑니다 — 녀 장짜리인데, 겁먹지 마세요. 1쪽에 쓰는 법이 다 적혀 있고, 2쪽에는 완성 예시가 통째로 있습니다. 같이 넘겨볼게요. (1쪽) 다섯 단계입니다 — 정하고, 쏟아내고, 묶고, 그리고, 문패 달고. 핵심은 순서예요. 처음부터 트리를 그리려고 하면 막힙니다. 먼저 3쪽에서 내 앱에 필요한 걸 생각나는 대로 다 쏟아내고 — 무슨 화면? 무슨 동작? 뭐가 남아야 해? — 그다음 비슷한 것끼리 묶어서 폴더 이름을 짓고, 그걸 4쪽에서 트리로 옮기는 거예요. (2쪽) 완성 예시입니다 — 점심 메뉴 뽑기 앱을 다섯 단계 전부 채워놨어요. 막막하면 이 페이지를 펴놓고 형식을 그대로 빌리세요. 베끼는 게 아니라 형식을 빌리는 겁니다 — 내용은 여러분 앱이니깐요. 합격 기준 세 개 — 하나, 최상위 폴더 3~6개. 둘, 모든 폴더에 '왜' 한 줄. 셋, 폴더마다 4겹 태그, 두 개 붙어도 되고 연결이 없으면 없어도 됩니다 — 예시에도 연결 칸이 비어 있죠, 비워도 되는 칸이 있다는 것도 설계예요. 다시 한번 — 폴더 3~6개, 모든 폴더에 왜, 4겹 태그. 그리고 제일 중요한 것 — 14차는 이 종이로 시작합니다. 반드시 가져오세요. 잘 그린 순서가 아니라, 이유를 쓴 순서로 좋은 설계입니다.

### 💡 강사 팁

설계지 1~2쪽을 실물로 넘기며 브리핑하는 게 핵심 — "쓰는 법이 종이에 있다"를 눈으로 확인시키면 숙제 문의가 급감한다. 소재를 못 정하는 사람에겐 "화면 3개 이하·한 문장 설명·혼자 써도 쓸모" 기준 + 작은 소재(물 마시기 기록·읽은 책 기록)를 조용히 권한다. 예시(점심 메뉴 뽑기)와 같은 소재를 골라도 막지 말 것 — 단 "예시와 다른 폴더 하나를 추가해보라"고 권한다.

### ⚠️ 주의

"다음 시간 이 종이로 시작합니다"를 명시적으로 못 박을 것 — 14차 오프닝(숙제 리뷰)의 성패가 여기서 갈린다.

→ 넘어가며: "마지막으로 — 오늘의 문장 한 번만 다시."

## 엔딩 — 지도를 들고 들어온 날

⌚ 4분 (156-160)

### ★ KEY POINT

오늘 처음으로 지도를 들고 개발 세계에 들어왔다. 14차 예고 — 그 구조 위에 화면을 그린다.

### ■ 화면

엔딩(다크) — "모든 파일에는 자리가 있고, 그 자리에는 이유가 있다" + 14차 예고.

### 💬 이렇게 말하세요

오늘 우리는 아무것도 만들지 않았습니다. 대신 지금까지 만들어온 모든 것의 속을 이해했어요. 이제 여러분은 어떤 프로젝트 폴더를 열어도 — 재료 목록을 찾고, 창고를 알아보고, 문패부터 읽을 수 있습니다. 지도를 들고 다니는 사람이 된 거예요. 다음 시간엔 그 구조 위에 화면을 그립니다 — 뼈대와 옷과 움직임, 홀의 인테리어를 배우는 날이에요. 설계지 꼭 챙겨 오시고요. 오늘의 문장으로 마칩니다 — 모든 파일에는 자리가 있고, 그 자리에는, 이유가 있다. 수고하셨습니다!

### 📌 수업 후 강사 체크리스트

① 숙제 안내가 명확했는가(합격 기준 2회 언급 완료 여부) ② 유난히 막힌 용어 기록(04 사전 보강 + 14차 한 줄 복습 대상) ③ 밀린 구간 기록(14차 타임라인 보정) ④ "코드 보여달라" 요청 여부(14차 수요 신호) ⑤ 랜딩 course-12 반영 확인.



## 부록 — 전체 지도 한 장

🕒 시간 배정 없음 (필요할 때)

### ★ KEY POINT

질문이 나오면 폴더를 여는 대신 이 장을 띄운다 — 강의 중 화면 전환(슬라이드 ↔ 탐색기) 없음.

### ■ 화면

루트+src 통합 트리 + 여정 요약 + 원칙 5 한 줄.

### 💡 쓰는 순간

① 숙제 브리핑 중 "그 파일은 어디 있어요?" 류 질문 ② 쉬는 시간·수업 후 질의응답 ③ 클로징 후 화면에 띄워두는 배경. 학생 질문별 답변 요령은 03\_구조지도\_강사원본 §4 FAQ 참조.

### ⚠ 주의

이 장을 본편에서 미리 쓰지 말 것 — 13·17·18의 단계적 공개(1층 → 2층 → 확대)가 무너진다. 본편이 끝난 뒤에만.